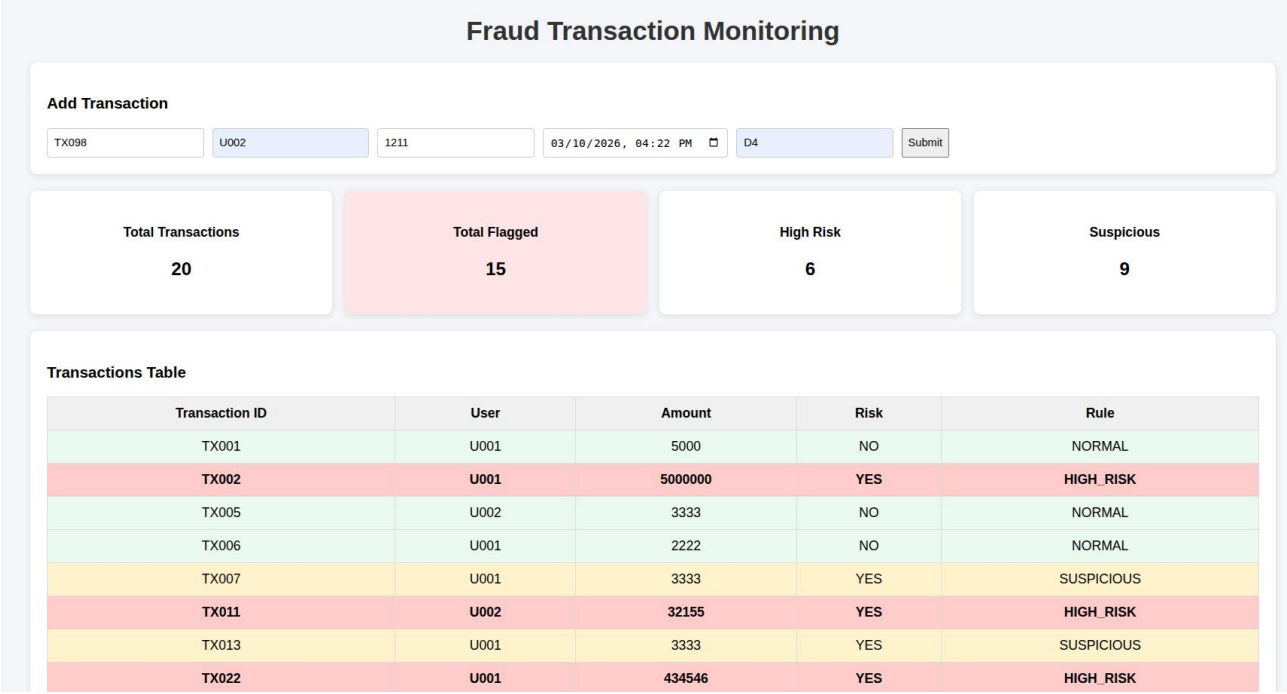
Mini Transaction Risk Monitoring System

1. Overview:

This project is small fraud detection system that identifies fraud transaction on specific rules.
its highlights these transactions and displays their counts on a dashboard



Transaction ID	User	Amount	Risk	Rule
TX001	U001	5000	NO	NORMAL
TX002	**U001**	**5000000**	**YES**	**HIGH_RISK**
TX005	U002	3333	NO	NORMAL
TX006	U001	2222	NO	NORMAL
TX007	U001	3333	YES	SUSPICIOUS
TX011	**U002**	**32155**	**YES**	**HIGH_RISK**
TX013	U001	3333	YES	SUSPICIOUS
TX022	**U001**	**434546**	**YES**	**HIGH_RISK**

2. Main Features:

- **Upload transaction form**
- **List all transaction,highlighting High Risk and Suspicious transactions**
- **Dashboard analytics showing total transactions and red-flagged counts**

3. Backend:

3.1. Transaction Form (post api)

- Submit transaction details via a form.
- Post **transaction** using the **POST/transactions** API.

POST {{base_url}} transactions

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
"transaction_id": "TX201",
"user_id": "U1001",
"amount": 25000,
"timestamp": "2026-02-23T10:00:00",
"device_id": "D7"
}
```

Body Cookies Headers (4) Test Results

{} JSON Preview Visualize

```
{
    "status": "success",
    "message": "new transaction is successfully added"
}
```

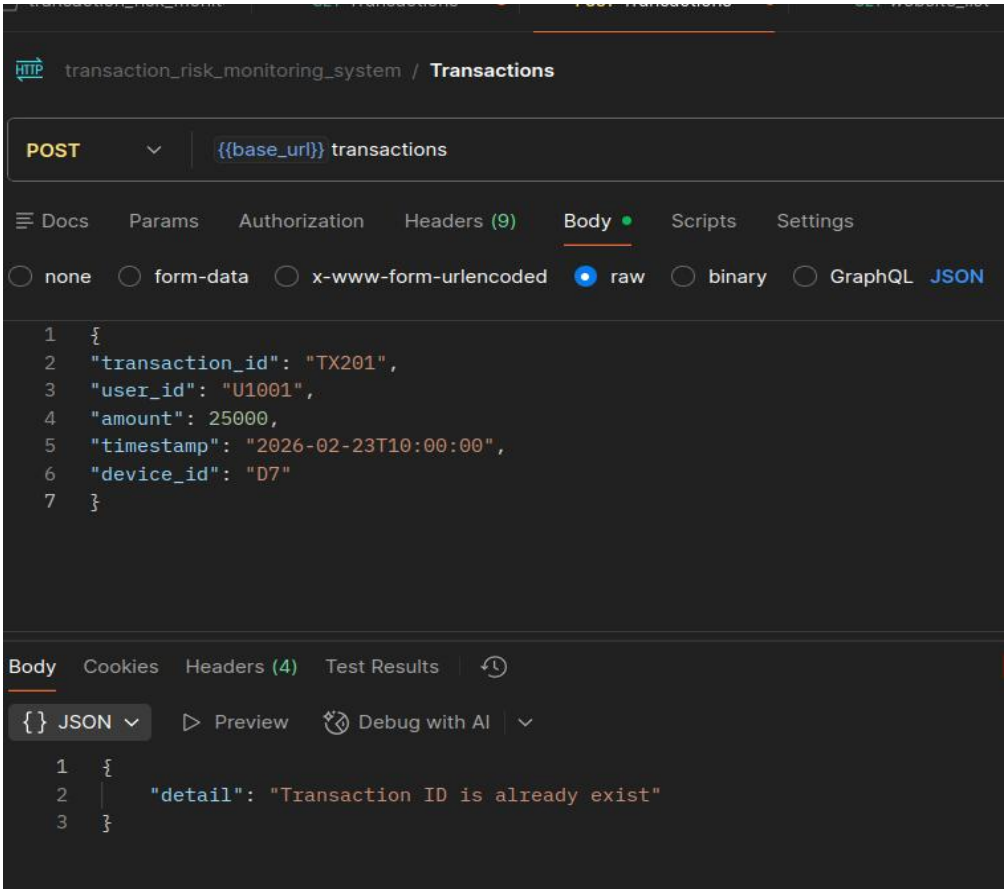
3.2.Validation Checks

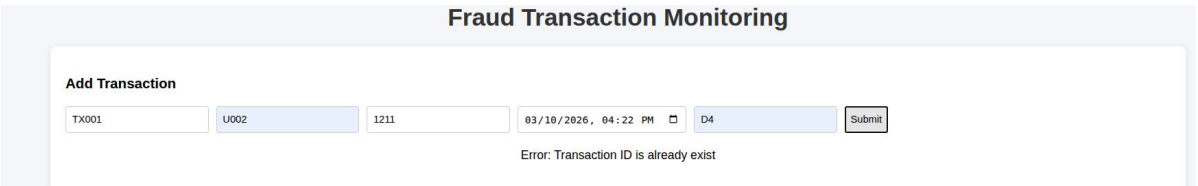
- Ensure all fields are filled if not its give .
- Verify the data types for each field.
- Validate that the transaction amount is greater than 0 before processing.
- If the amount is 0 or negative, return an error like (invalid transaction amount. Amount must be greater than 0)
- Validate that both user**_id,transaction_id** and **device_id** have a minimum length of 1 character
- If either field is empty, return an error

```python
class TransactionCreate(BaseModel):
    transaction_id: str = Field(..., min_length=1)
    user_id: str = Field(..., min_length=1)
    amount: float = Field(..., gt=0)
    timestamp: datetime
    device_id: str = Field(..., min_length=1)
```

3.3. Duplicate Transaction Check

- If the transaction is valid, check whether the transaction ID already exists in the system.
- If a duplicate transaction is found, return:
  "Transaction ID is already exists"





3.4. Fraud Rule Evaluation:

If the transaction is not a duplicate, evaluate it based on the following rules:

Rule 1: High Risk

- If the transaction amount is greater than **2000**, mark it as **High Risk**.

Rule 2: Suspicious Activity

- If a user performs more than **three transactions**, mark the transaction as **Suspicious**.

```python
import schemas
def apply_fraud_rules(cursor, tx: schemas.TransactionCreate):
    """
    Apply fraud rules to the transaction.
    Rule1: amount > 20,000 -> HIGH_RISK
    Rule2: more than 3 transactions by same user -> SUSPICIOUS
    """
    if tx.amount > 20000:
        return True, "HIGH_RISK"

    cursor.execute("SELECT COUNT(*) FROM transaction_tracking WHERE user_id=%s", (tx.user_id,))
    count = cursor.fetchone()[0]
    if count >= 3:
        return True, "SUSPICIOUS"

    return False, None
```

3.5. Store Transaction

- Store all transactions in the database.
- Save the corresponding flags for **High Risk** and **Suspicious** transactions.
- Now, since transaction volume is manageable, all transactions are stored in one main table.

```
mysql> select * from transaction_tracking;
+----+----------------+---------+---------------------+-------------+------------+-----------+----------------+---------------------+
| id | transaction_id | user_id | transaction_time    | device_info | amount     | risk_flag | rule_triggered | created_at          |
+----+----------------+---------+---------------------+-------------+------------+-----------+----------------+---------------------+
|  3 | TX001          | U001    | 2026-02-23 10:00:00 | D1          |    5000.00 |         0 | NULL           | 2026-02-28 07:06:07 |
|  4 | TX002          | U001    | 2026-02-23 10:00:00 | D1          | 5000000.00 |         1 | HIGH_RISK      | 2026-02-28 07:07:51 |
|  5 | TX005          | U002    | 2026-03-11 00:00:00 | D4          |    3333.00 |         0 | NULL           | 2026-03-01 00:00:17 |
|  7 | TX006          | U001    | 2026-03-18 00:02:00 | D4          |    2222.00 |         0 | NULL           | 2026-03-01 00:02:34 |
|  8 | TX007          | U001    | 2026-03-18 00:03:00 | D4          |    3333.00 |         1 | SUSPICIOUS     | 2026-03-01 00:03:13 |
|  9 | TX011          | U002    | 2026-03-04 06:55:00 | D4          |   32155.00 |         1 | HIGH_RISK      | 2026-03-01 06:55:44 |
```

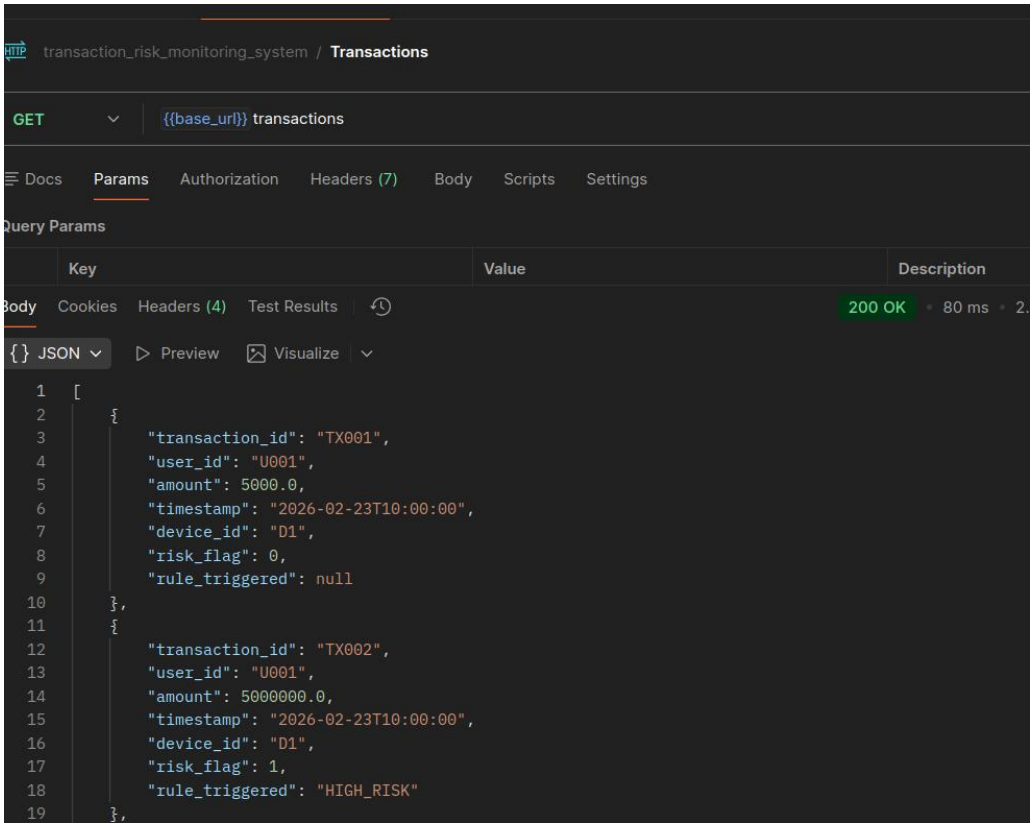
3.5.1 Indexed DB Fields

- user_id has a separate index (idx_user_id), which speeds up queries filtering transactions by user which help during finding suspicious transationx
- transaction_id has a unique constraint, so it is automatically indexed.

```
----------+
| transaction_tracking | CREATE TABLE `transaction_tracking` (
  `id` int NOT NULL AUTO_INCREMENT,
  `transaction_id` varchar(50) NOT NULL,
  `user_id` varchar(50) NOT NULL,
  `transaction_time` datetime NOT NULL,
  `device_info` varchar(50) DEFAULT NULL,
  `amount` decimal(12,2) DEFAULT NULL,
  `risk_flag` tinyint(1) DEFAULT '0',
  `rule_triggered` varchar(50) DEFAULT NULL,
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `transaction_id` (`transaction_id`),
  KEY `idx_user_id` (`user_id`)
) ENGINE=InnoDB AUTO_INCREMENT=24 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
```

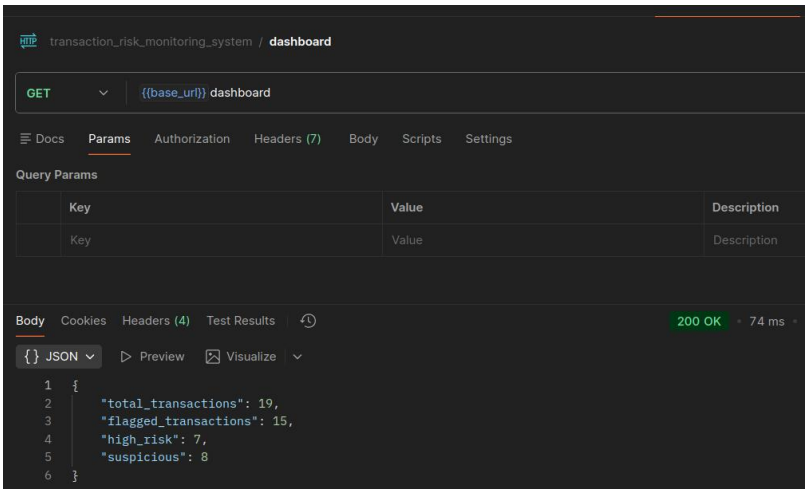
3.6. Get All Transactions

- Retrieve all transactions using the appropriate API endpoint.
- Retrieve all transactions using the **GET** /transactions API.



3.7. Get Dashboard Statistics

- Retrieve dashboard analytics, including total transactions, high-risk transaction count, and suspicious high_risk count
- Retrieve transaction statistics using the **GET** /dashboard API.

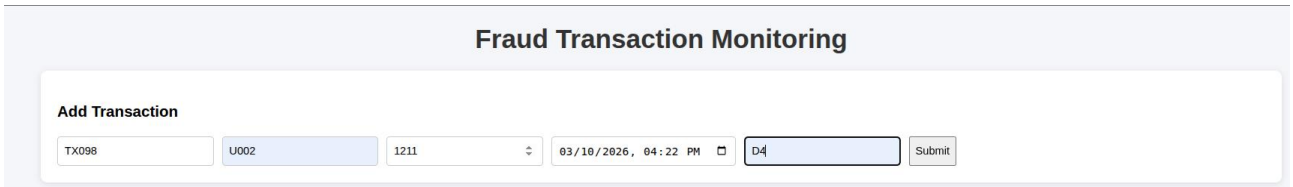


4. Frontend:

4.1 Upload Transactions Form

Make user input which have following columns

- transaction_id
- user_id
- amount
- timestamp
- device_id



Upload Form

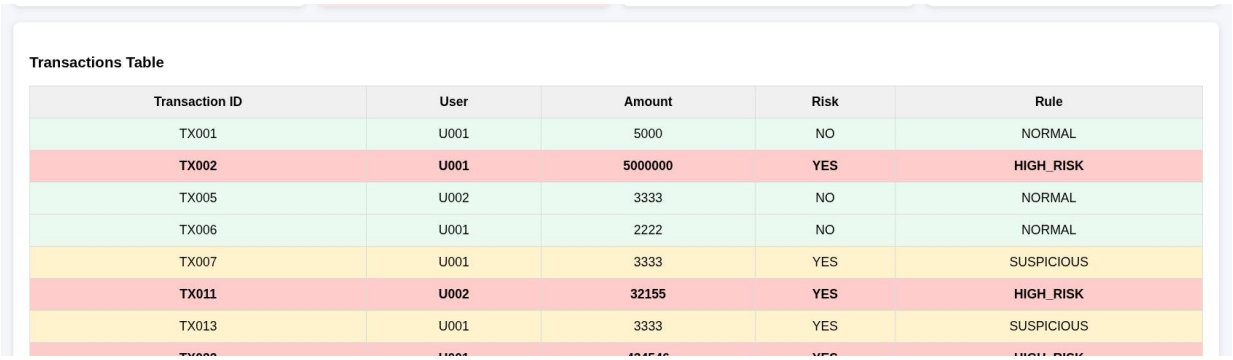
4.2 Transactions Table

4.2.1 List all transactions

- columns:Transaction ID,User,Amount,Risk,Rule

4.2.2 Highlight flagged ones

- high-risk-row -> background-color: #ffcccc;
- suspicious-row -> background-color: #fff3cd;
- normal-row -> background-color: #eafaf1;



Transactions Table

Transaction ID	User	Amount	Risk	Rule
TX001	U001	5000	NO	NORMAL
TX002	**U001**	**5000000**	**YES**	**HIGH_RISK**
TX005	U002	3333	NO	NORMAL
TX006	U001	2222	NO	NORMAL
TX007	U001	3333	YES	SUSPICIOUS
TX011	**U002**	**32155**	**YES**	**HIGH_RISK**
TX013	U001	3333	YES	SUSPICIOUS

Transaction table

5. Dashboard Section

Dashboard that provides clear visibility:

- total transactions count
- total flagged transaction count
- high risk transaction count
- suspicious transaction count

Dashboard

6. To handle 1 million transactions per day:

The following optimizations will be implemented:

6.1 Batch Inserts with Kafka

- Process transactions in batches instead of single inserts.
- Reduces database load and improves write performance

6.2. Redis Caching

- Use Redis for Frequently accessed data

6.3. Database Optimization

Now, since transaction volume is manageable, all transactions are stored in one main table.

Apply the following structure:

6.3.1. Create transaction_flag Table

- Create a *transaction_flag* table with transaction_id as a foreign key to maintain data integrity.
- It keeps the main transaction_tracking table lighter, improving performance and scalability

6.3.2. Create Stats_counter table

- Create a stats_counter table to store pre-calculated counters, avoiding heavy COUNT() and SUM() queries on large datasets**.**
- When a rule is triggered, increment the counter immediately, eliminating the need for expensive aggregation queries later

6.4. Asynchrouous Function

Use async functions to handle multiple operations concurrently for better performance

Apply async to:

- **Database operations** as a result non-blocking queries
- **API handling** handle multiple requests simultaneously
- **Rule processing** running transaction process in parallel